Reproducibility Guide — SafeGen-SBP (v2.2.0)

Recorded environment

Final verification run for **SafeGen-SBP v2.2.0**: Python 3.12.3, NumPy 2.1.3, SciPy 1.14.1 (Linux environment). The exact complete standard output is preserved in `../results/final_verification.log`. Wall-clock execution times are environment-specific and are not expected to reproduce bit-for-bit across machines.

Reproduce from a clean environment

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python safegen_sbp.py | tee results/final_verification.log
```

Pinned direct dependencies in `requirements.txt`: `numpy==2.1.3` and `scipy==1.14.1`. All stochastic benchmarks use `numpy.random.default_rng(seed)` across the fixed seed list `[42, 43, 44, 45, 46]`.

Verification test suite (8 tests)

Running `python safegen_sbp.py` first executes eight asserted unit and integration tests:

1. **Test 1:** 2D projection `[4, 5] -> [1.5, 2.5]` onto `x1 + x2 <= 4`.
2. **Test 2:** Bitwise preservation of an already-feasible point `[1, 2]`.
3. **Test 3:** Explicit `ValueError` rejection for zero, `NaN`, and `Inf` normal vectors, incompatible vector/matrix dimensions, nonpositive `max_iters`, and negative or non-finite tolerances.
4. **Test 4:** Squared-norm policy cutoff verification (`||g||_2 = 1e-6` with `g.T @ g = 1e-12 > 1e-14` is accepted; `||g||_2 = 1e-8` with `g.T @ g = 1e-16 <= 1e-14` is rejected) and pedagogical reconstruction of the `+1e-12` denominator offset (`[2, 5] -> [1.5, 5]` vs. exact `[1, 5]`).
5. **Test 5:** Large-scale 2D normal (`g = [1e6, 0]`, `beta = 2e6`) projecting `[4, 3]` to `[2, 3]`.
6. **Test 6:** Multi-constraint Dykstra (`step_tol=1e-12`, `feas_tol=1e-12`) compared against SciPy SLSQP (`ftol=1e-12`), asserting an `L_inf` solution gap `< 1.1e-9` (measured `1.152855588771e-12`).
7. **Test 7:** Contradictory 2D constraints (`x1 + x2 <= 2` and `-x1 - x2 <= -5`) over 50 cycles, yielding `max_signed_violation = 3.0`, `is_feasible = False`, and `x = None` without claiming a formal certificate of infeasibility.
8. **Test 8:** Iteration-budget exhaustion (`max_iters = 1`, `step_tol = 1e-15`) kept distinct from safety certification (`budget_exhausted = True`, `converged = False`, `is_feasible = False`, `max_violation = 2.32`).

Benchmark specifications

- **Benchmark 1 (50D single half-space):** For each of the 5 seeds, draw one random normal vector in $\mathbb{R}^{50}$ normalized to $\|g\|_2 = 10^{-6}$ with $\beta = 0$, and draw 1,000 input vectors from $\mathcal{N}(0.5, 1.0)$ (5,000 samples total). Reported violation rates are the mean and population standard deviation (`ddof=0`) across the five per-seed rates; reported max violations are the mean of the five per-seed maxima (distinct from a single global maximum). Violation counting thresholds in the code are >

0 for raw inputs, > 1e-15 for the reconstructed +1e-12 offset model, and > 1e-12 for exact projection. Even under the strictest threshold, exact projection residuals remain around 1e-21.

- **Benchmark 2 (20D polytope vs. SLSQP):** Dimension $n = 20$ with $m = 21$ constraints: $\sum_{i=1}^{20} x_i \leq 15$ and $-x_i \leq 0$ ($x_i \geq 0$). For each of the 5 seeds, draw 200 inputs from $\mathcal{N}(1, 2)$ (1,000 paired samples total). Dykstra uses `step_tol=1e-8`, `feas_tol=1e-7`, `max_iters=500`; SLSQP uses `ftol=1e-12`, `maxiter=500`. Feasibility/success counters (`d_ok` and `s_ok`) are incremented independently. Timing uses a single sequential `time.perf_counter()` pass per sample without warm-up or repeated trials (`13.620408 ms/sample` for Dykstra vs. `0.475233 ms/sample` for SLSQP; ratio `0.034891x`).